

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Case Study

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE CODE: CSA1516

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)*

Assessment Tool Weightage: 20%

Dave's Taxonomy: Precision (Level 3)
Question Count: 3

Total Marks: 30

Code of Conduct

I R.Jaideep(192525441)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: R.Jaideep

Assessment Tasks

Case Study

1. Case 1: A food delivery company wants to deploy a cloud-native backend with web APIs, browser clients, and shared cloud storage for receipts and delivery images. Propose the architecture and explain the role of each component.
2. Case 2: A media startup must decide between SAN-backed storage, object storage, and serverless processing for video upload workflows. Analyze the case and recommend a deployment model.
3. Case 3: A smart city platform needs low-latency decision making near sensors while still using the cloud for analytics. Analyze the use of edge computing and fog computing in this environment.

Case Study Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 – Developing	Level 1 - Beginning
Understanding of Case	25%	Clearly identifies all technical and business requirements	Identifies most requirements	Basic understanding only	Poor understanding of case
Application of Concepts	25%	Applies suitable cloud concepts and tools effectively	Good application with minor issues	Partial application	Weak or incorrect application
Analysis	20%	Strong analysis with clear trade-off reasoning	Reasonable analysis	Limited analysis	Minimal analysis

Solution Design	20%	Solution is feasible, practical, and well justified	Reasonable solution with minor gaps	Basic solution	Unclear solution
Clarity	10%	Well-structured and easy to follow	Mostly clear	Somewhat unclear	Poorly organized

1.Cloud-Native Architecture for a Food Delivery Company

A food delivery company can use a **cloud-native three-tier architecture** to provide scalable, reliable, and secure food ordering and delivery services. The architecture will contain browser clients, web/API services, cloud databases, shared cloud storage, load balancing, authentication, and monitoring services. These components will work together to process customer requests, manage restaurant and delivery information, and store receipts and delivery images.

Browser Clients

Customers, restaurants, and delivery partners will access the application through **web browsers**. The browser client will provide interfaces for user registration, restaurant search, food ordering, payment status, order tracking, and delivery updates. It will communicate with backend services through secure **HTTPS-based REST APIs** rather than directly accessing databases or cloud storage.

Load Balancer

A **cloud load balancer** will receive incoming requests from browser clients and distribute them among multiple backend API instances. This prevents one server from becoming overloaded and improves application availability. If one API instance fails, the load balancer can redirect requests to healthy instances.

Web/API Layer

The backend will be implemented as **cloud-native web API services**. APIs will handle operations such as user authentication, restaurant information, menu management, order processing, payment status, delivery tracking, and notifications. The services can be deployed using containers and automatically scaled according to traffic. Separating APIs into independent services will make the system easier to develop, update, and maintain.

Authentication and Authorization

An identity service will manage user login and access permissions. Customers, restaurant staff, delivery partners, and administrators will receive appropriate roles. **OAuth 2.0 or JWT-based authentication** can be used to securely access APIs. Authorization will ensure that users can access only the resources permitted for their roles.

Cloud Database

A managed cloud database will store structured information such as customer details, restaurant data, menus, orders, delivery status, and transaction records. A relational database can be used where strong consistency and relationships are important.

Database backups, replication, and automated recovery will help protect important business information.

Shared Cloud Storage

Object-based cloud storage will be used for **receipts, food images, delivery photographs, invoices, and other large files**. Instead of storing these files directly inside the database, the API will upload them to cloud storage and store their secure object references in the database. This approach provides high scalability and reduces database storage requirements. Access to sensitive files can be controlled using permissions and temporary secure URLs.

Cache and Messaging Services

A cloud cache can temporarily store frequently requested information such as restaurant menus and popular food items, reducing database load and improving response time. A message queue can handle background tasks such as sending order notifications, generating receipts, processing images, and updating delivery information. This allows the main API to respond quickly without waiting for every background operation to complete.

Auto-Scaling and Monitoring

The cloud platform will automatically increase or decrease the number of API instances based on CPU usage, request rate, or other performance metrics. Monitoring services will track API response time, errors, storage usage, database performance, and system health. Alerts will notify administrators when abnormal conditions occur.

Overall Architecture Flow

Browser Client → Load Balancer → Web/API Services → Database / Cloud Storage / Cache / Message Queue

The browser will send requests through HTTPS to the load balancer. The load balancer will forward requests to available API services. APIs will authenticate users, process business logic, retrieve information from the database, and store large files in shared cloud storage. Background activities will be handled through messaging services, while monitoring and auto-scaling will maintain system performance.

Thus, this cloud-native architecture will provide **scalability, high availability, security, efficient storage, and faster response** for the food delivery company. It will also allow the company to handle sudden increases in orders during weekends, festivals, and promotional events without requiring major changes to the infrastructure.

Case 2: Deployment Model for Video Upload Workflows

A media startup that handles video uploads must choose a storage and processing architecture that can support **large file sizes, high upload traffic, scalability, reliability, and cost efficiency**. The three important technologies being considered are **SAN-backed storage, object storage, and serverless processing**. Each technology provides different capabilities. After analyzing the requirements, the recommended deployment model is a **cloud-native architecture that uses object storage as the primary video repository and serverless processing for event-driven video workflows**. SAN storage can be used selectively for specialized high-performance workloads.

1. SAN-Backed Storage

A **Storage Area Network (SAN)** provides high-performance block-level storage that can be connected to application servers. It behaves like a high-speed disk system and provides predictable performance and low latency. SAN storage is commonly used for databases, enterprise applications, and workloads that require frequent read and write operations.

For a media startup, SAN can provide fast access to video files during active processing. For example, a video transcoding server could temporarily use SAN storage when it needs very high-speed read/write operations. However, using SAN as the primary storage for every uploaded video may not be economical.

Video files are generally large and can accumulate rapidly as the number of users increases. Expanding SAN capacity requires additional infrastructure and administration. The startup may therefore face higher hardware, maintenance, and operational costs. SAN is also less flexible than cloud object storage when storage requirements change rapidly.

2. Object Storage

Object storage is the most suitable option for storing large numbers of video files. Instead of storing files as traditional blocks, object storage stores each video as an object together with metadata and a unique identifier.

The major advantage of object storage is **massive scalability**. The startup does not need to purchase and maintain physical storage whenever the number of videos increases. Cloud object storage can expand according to demand.

Object storage also provides high durability and can support multiple storage classes. Frequently accessed videos can remain in a standard storage tier, while older videos can automatically move to cheaper archival tiers. Lifecycle policies can also delete temporary files after a specified period.

Another important advantage is that object storage can be accessed through standard web protocols. This makes it convenient for applications, APIs, content delivery networks, and media-processing services.

Therefore, object storage should be selected as the **primary storage layer for raw videos, processed videos, thumbnails, subtitles, and other media assets**.

3. Serverless Processing

Storing videos is only one part of the workflow. Uploaded videos usually require additional processing such as **format validation, metadata extraction, thumbnail generation, compression, transcoding, and quality conversion.**

A traditional approach would require continuously running servers for these operations. This can result in wasted resources when there are few uploads. During peak periods, the servers may also become overloaded.

Serverless processing provides a better solution for workloads that are triggered by events. When a video is uploaded to object storage, an event can automatically trigger a serverless function or workflow.

For example:

Video Upload → Object Storage → Event Trigger → Serverless Function → Video Processing → Processed Storage

The serverless function can validate the uploaded file and start a processing workflow. The workflow can generate thumbnails, extract metadata, and invoke a suitable media-transcoding service.

Serverless computing automatically scales according to the number of events. If 10 videos are uploaded, only a small number of processing tasks may be required. If thousands of videos are uploaded during a promotional campaign, the cloud platform can scale processing capacity automatically.

This approach reduces infrastructure management and follows a **pay-per-use model**, making it particularly attractive to a startup.

4. Recommended Deployment Model

The recommended deployment model is a **combination of object storage and serverless processing**, supported by a CDN, database, API layer, and monitoring services.

The overall architecture can be represented as:

User → Web/Mobile Application → API Gateway → Object Storage → Event Trigger → Serverless Processing → Processed Object Storage → CDN → Users

The process begins when a user selects and uploads a video through the web or mobile application. The application sends the upload request through an API layer. Instead of sending the complete video through the application server, the system can provide a secure temporary upload URL. The client can then upload the video directly to object storage.

After successful upload, an event is generated. This event activates the serverless processing workflow. The workflow can perform validation, extract metadata, generate thumbnails, and start transcoding into different formats and resolutions.

The processed videos are stored in separate object-storage locations. For example, the startup can maintain separate areas for **original videos, processed videos, thumbnails, and temporary files**.

A database can store information about each video, such as its owner, file location, duration, processing status, and upload timestamp. The application can use this information to display the user's video library.

Finally, a **Content Delivery Network (CDN)** can distribute processed videos to viewers. The CDN caches frequently requested content at locations closer to users, reducing latency and improving playback performance.

5. Role of SAN in the Recommended Architecture

Although object storage is recommended as the primary storage system, SAN should not be completely rejected. It can be useful when the startup has specific processing workloads requiring high-speed block storage.

This creates a practical architecture where:

- **Object storage** handles permanent and large-scale media storage.
- **Serverless processing** handles event-driven and variable processing workloads.
- **SAN storage** handles specialized high-performance temporary processing when required.
- **CDN** provides fast global delivery of processed videos.
- **Database** stores metadata and application information.

6. Security and Reliability

Security is important because uploaded videos may contain private or copyrighted content. Object storage should use access-control policies so that users cannot access other users' files. Encryption should be applied to data both at rest and during transmission.

Temporary upload and download URLs can provide controlled access without exposing storage credentials. Authentication and authorization should be implemented through the application's identity-management system.

Reliability can be improved using automatic backups, replication, versioning, and lifecycle policies. Monitoring services should track upload failures, processing errors, storage usage, and serverless execution performance.

7. Cost and Scalability Analysis

From a cost perspective, using SAN for the entire video library would require significant investment as the number of videos increases. Object storage is more suitable because the startup can scale storage without purchasing physical infrastructure.

Serverless processing further reduces costs because the company does not need to maintain continuously running processing servers. Resources are consumed primarily when processing tasks are executed.

Case 3: Edge Computing and Fog Computing in a Smart City Platform

A smart city platform collects data from a large number of sensors installed across roads, traffic signals, parking areas, surveillance systems, public transport, waste-management systems, and environmental monitoring stations. Since these sensors continuously generate large amounts of data, sending every piece of information directly to a centralized cloud can cause **network delay, high bandwidth consumption, and slower decision-making**. Therefore, the smart city should use a combination of **edge computing, fog computing, and cloud computing**.

1. Need for Edge and Fog Computing

Traditional cloud computing sends sensor data to a centralized cloud data center for processing. Although cloud computing provides high processing power and large-scale storage, it may not be suitable for time-sensitive applications.

For example, if a traffic sensor detects an accident, the system may need to immediately change traffic signals or send an alert to nearby vehicles. Sending the information to a distant cloud, processing it, and returning the decision can introduce unnecessary latency.

Edge and fog computing solve this problem by moving computation closer to the location where data is generated.

Edge computing processes data directly at or very close to the sensors and IoT devices, while **fog computing** provides an intermediate computing layer between edge devices and the centralized cloud.

2. Role of Edge Computing

Edge computing places processing capabilities near the data source. In a smart city, edge devices can be installed at traffic intersections, cameras, buildings, parking systems, and environmental monitoring stations.

For example, a traffic camera can use an edge device to identify vehicle congestion. Instead of sending the complete video stream to the cloud, the edge device can analyze the video locally and send only important information such as vehicle count, congestion level, or accident detection.

The major advantage is **very low latency**. Critical decisions can be made within milliseconds or seconds without waiting for cloud processing.

Edge computing can be used for:

- Real-time traffic signal control.
- Accident and emergency detection.
- Smart parking availability.
- Security camera event detection.
- Air-quality monitoring.
- Industrial and utility monitoring.

- Immediate emergency alerts.

Edge computing also reduces bandwidth usage because only processed information or important events need to be transferred to higher-level systems.

3. Role of Fog Computing

Fog computing creates an intermediate processing layer between edge devices and the cloud. Instead of every sensor communicating directly with the cloud, groups of nearby sensors can communicate with a **fog node or gateway**.

For example, sensors at several traffic intersections in one city zone can send information to a nearby fog node. The fog node can combine and analyze information from multiple intersections and make regional decisions.

Fog computing is therefore useful when decisions require information from multiple edge devices.

The fog layer can perform:

- Data aggregation.
- Local analytics.
- Temporary data storage.
- Device management.
- Network traffic management.
- Regional decision-making.
- Filtering and preprocessing before cloud transmission.

This architecture reduces the amount of raw sensor data sent to the cloud.

4. Difference Between Edge and Fog Computing

Edge and fog computing are related but operate at different levels.

Feature	Edge Computing	Fog Computing
Processing location	Directly near sensors/devices	Intermediate layer between edge and cloud
Latency	Very low	Low
Data scope	Individual device or local sensor	Multiple nearby devices
Main function	Immediate decisions	Aggregation and regional processing
Example	Detecting accident using camera	Coordinating traffic across several intersections
Cloud dependency	Low for immediate actions	Connects edge systems with cloud

Thus, edge computing is mainly responsible for **immediate local decisions**, while fog computing performs **broader local or regional coordination**.

5. Smart City Architecture

A suitable architecture can be represented as:

Sensors/IoT Devices → Edge Nodes → Fog Nodes → Cloud Platform → Central Analytics

At the first level, sensors collect information from the physical environment. Edge devices process time-sensitive information locally. The resulting data is forwarded to fog nodes for aggregation and regional analysis.

The fog layer then sends selected and processed data to the cloud. The cloud provides large-scale storage, historical analytics, machine learning, reporting, and centralized management.

For example, traffic sensors may detect vehicle movement at individual intersections. Edge devices can immediately detect congestion. Fog nodes can combine information from several intersections and optimize traffic flow across an entire area. The cloud can analyze months or years of traffic data to identify long-term congestion patterns and improve city planning.

6. Advantages of Edge Computing

The most important advantage of edge computing is **low latency**. Critical applications do not need to wait for a remote cloud server.

It also improves reliability. If the internet connection to the cloud temporarily fails, edge devices can continue performing essential local operations.

Edge processing reduces network bandwidth requirements because raw sensor data does not always need to be transmitted to the cloud. This is particularly important for high-volume data sources such as surveillance cameras.

It can also improve privacy because sensitive data can be analyzed locally and only necessary information can be transmitted to the cloud.

7. Advantages of Fog Computing

Fog computing provides greater processing capability than individual edge devices while remaining closer to the sensors than a centralized cloud.

It is useful for **regional decision-making**. For example, a fog node can receive data from multiple traffic signals, parking sensors, and cameras in one city zone and coordinate their operation.

Fog computing also reduces cloud workload by filtering, aggregating, and preprocessing data before transmission. This allows the cloud to focus on complex analytics rather than processing every raw sensor reading.

8. Role of Cloud Computing

Cloud computing remains an important part of the smart city architecture. The cloud should be used for tasks that do not require immediate responses.

Examples include:

- Long-term sensor-data storage.
- Historical traffic analysis.
- Machine-learning model training.
- City-wide dashboards.
- Predictive maintenance.
- Energy-consumption analysis.
- Population and transportation analytics.
- Centralized reporting.

The cloud can also receive information from different fog regions and provide city-wide insights.